Codebase Orientation

Appendix: Codebase Orientation Checklist

Phase 1: First Contact

1. Repository Overview

Clone the repo and get it running locally. Document every step, including anything that was not in the README.

Topic	README	What actually happened
Database	docker-compose up -d, Postgres on 5432	Used docker-compose.local.yml, Postgres on 5433 to avoid conflict with local Postgres
DATABASE_URL	Port 5432	Port 5433 for docker-compose.local.yml
Order	Seed, then migrate	Migrate, then seed (schema before data)
Login	Not mentioned	Stopped setting VITE_API_URL so the frontend uses the Vite proxy and avoids cross-origin cookie blocking
build:shared	Not mentioned	Ran before pnpm dev on a clean setup
Ports	Fixed 5173, 3000	dev.sh uses the next free ports (e.g. 5174/3001) when those are in use
Corrupted install	Not mentioned	Required removing node_modules and pnpm-lock.yaml, then reinstalling

2. Read every file in the docs/ folder. Summarize the key architectural decisions in your own words.

- **Everything is a document.** All content is treated as a document in order to eliminate the need for separate tables and schema migrations. Documents (wikis, issues, programs, projects, weeks, people, weekly plans, weekly retros) are differentiated by `document_type` field.
- **Application Architecture.** Node.js + Express, React + Vite, PostgreSQL with raw SQL, shadcn/ui (tailwind + radix). Architecture prioritizes well understood and battle tested tools over cutting edge. Offline tolerant, server is source of truth. Single process and repo to keep deployment simple and designed as a department level tool.

- **Realtime Collaboration.** TipTap editor + Yjs for conflict-free collaborative editing. Yjs is persisted to PostgreSQL and cached client-side in indexedDB for instant loading. Shared editor is used for all document types, not separate editor implementations.
- **Document Associations through junction table.** Organizational relationships use a document associations junction table instead of direct foreign key columns, and legacy columns were dropped.
- **Government Compliance as a Design Constraint.** Section 508/WCAG 2.1 AA accessibility is mandatory. PIV/CAC auth is primary authentication. CloudWatch only observability.
- Read the shared/ package. What types are defined? How are they used across the frontend and backend?
    - **Shared Package Types** — The @ship/shared package ("@ship/shared": "workspace:*" in both api/ and web/ package.json) exports ~53 types organized across 5 files:

File-by-File Inventory

- **constants.ts** — HTTP status codes, error codes, session timeouts
    - HTTP_STATUS — object with standard codes (200, 400, 401, 404, 500, etc.)
    - ERROR_CODES — semantic error strings (VALIDATION_ERROR, SESSION_EXPIRED, etc.)
    - SESSION_TIMEOUT_MS — 15 minutes (government standard)
    - ABSOLUTE_SESSION_TIMEOUT_MS — 12 hours (NIST SP 800-63B-4 AAL2)
- **types/user.ts** — Single User interface (id, email, name, isSuperAdmin, lastWorkspaceId, timestamps)
- **types/api.ts** — Generic response envelope
    - `ApiResponse<T>` — { success, data?, error? }
    - ApiError — { code, message, details? }
- **types/document.ts** — The bulk of the package (~48 exports):
    - Type aliases: DocumentType (10 values: wiki, issue, program, project, sprint, person, weekly_plan, weekly_retro, standup, weekly_review), IssueState (7 states), IssuePriority, BelongsToType, ApprovalState, WeekStatus, ICEScore, AccountabilityType, IssueSource, DocumentVisibility
    - Property interfaces: One per document type — IssueProperties, ProgramProperties, ProjectProperties, WeekProperties,

PersonProperties, WikiProperties, WeeklyPlanProperties, WeeklyRetroProperties, StandupProperties, WeeklyReviewProperties
    - Supporting interfaces: BelongsTo, ApprovalTracking, PlanHistoryEntry, CascadeWarning, IncompleteChild
    - Base Document interface: 25+ fields (id, workspace_id, document_type, title, content, yjs_state, properties, ticket_number, timestamps, lifecycle dates, conversion tracking)
    - 10 typed variants: WikiDocument, IssueDocument, ProgramDocument, etc. — narrow the document_type and properties fields
    - Utilities: computeICEScore() function, DEFAULT_PROJECT_PROPERTIES constant
  - **types/workspace.ts** — Workspace, membership, and audit types
    - Workspace, WorkspaceMembership, WorkspaceInvite, AuditLog, WorkspaceWithRole, MemberWithUser
  - **types/auth.ts** — Currently empty (auth types defined locally in each package)

How They're Used Across Packages

Export	API (13 files)	Web (13 files)
HTTP_STATUS + ERROR_CODES	8 route files for validation/error responses	—
SESSION_TIMEOUT_MS / ABSOLUTE_SESSION_TIMEOUT_MS	Auth middleware, auth routes, collaboration server, CAIA auth, tests (6 files)	useSessionTimeout hook for client-side warning UI (1 file)
BelongsTo / BelongsToType	—	Sidebars, chips, editor, issue list (5 files)
ApprovalTracking	—	ApprovalButton, PropertiesPanel, WeekSidebar, ProjectSidebar (4 files)
computeICEScore + DEFAULT_PROJECT_PROPERTIES	projects.ts, dashboard.ts (server-side computation)	useProjectsQuery, ProjectSidebar, ProjectDetailsTab (client-side display)
DocumentType / IssueState / IssuePriority	—	contextMenuActions (filtering/rendering)

Export	API (13 files)	Web (13 files)
AccountabilityType	accountability service (1 file)	—

Pattern summary: Constants lean toward API use (error handling, session enforcement). Document model types lean toward web use (rendering, UI state). Business logic like ICE scoring is shared equally.

Create a diagram of how the web/, api/, and shared/ packages relate to each other.

Package Relationship Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│                        pnpm-workspace.yaml                       │
│                    packages: [api, web, shared]                  │
└──────────────────────────────────────────────────────────────────┘

                        ┌──────────────────────┐
                        │    shared/           │
                        │    @ship/shared      │
                        │                      │
                        │  types/              │
                        │  ├─ document.ts ─────┼──── DocumentType, IssueState,
                        │  │                   │     Document, *Properties,
                        │  │                   │     BelongsTo, ApprovalTracking,
                        │  │                   │     computeICEScore()
                        │  ├─ user.ts ─────────┼──── User
                        │  ├─ workspace.ts ────┼──── Workspace, WorkspaceMembership,
                        │  │                   │     AuditLog, MemberWithUser
                        │  ├─ api.ts ──────────┼──── ApiResponse<T>, ApiError
                        │  └─ auth.ts          │     (empty)
                        │                      │
                        │  constants.ts ───────┼──── HTTP_STATUS, ERROR_CODES,
                        │                      │     SESSION_TIMEOUT_MS,
                        │                      │     ABSOLUTE_SESSION_TIMEOUT_MS
                        └──────────────────────┘
                                   │
                        ┌──────────┴──────────┐
        depends on      │                     │      depends on
       workspace:*      │                     │     workspace:*
                        ▼                     ▼
┌──────────────────────┐      ┌──────────────────────────┐
│     api/             │      │     web/                 │
│     Express backend  │      │     React + Vite frontend│
│                      │      │                          │
│  USES FROM SHARED:   │      │  USES FROM SHARED:       │
│                      │      │                          │
│  ● HTTP_STATUS &     │      │  ● DocumentType, IssueState,
│    ERROR_CODES       │      │    IssuePriority         │
│    → 8 route files   │      │    → context menus, filters
│    → error responses │      │                          │
│                      │      │  ● BelongsTo, BelongsToType
│  ● SESSION_TIMEOUT_MS│      │    → sidebars, chips, editor
│    → auth middleware │      │                          │
│    → session validation      │  ● ApprovalTracking      │
│    → WebSocket auth  │      │    → approval buttons,   │
│                      │      │      properties panels   │
│  ● computeICEScore() │      │                          │
│    DEFAULT_PROJECT_PROPS     │  ● computeICEScore()     │
│    → project routes  │      │    → project sidebar/tabs
│    → dashboard route │      │    → query hooks         │
│                      │      │                          │
│  ● AccountabilityType│      │  ● SESSION_TIMEOUT_MS    │
│    → accountability svc      │    → useSessionTimeout hook
│                      │      │      (idle warning UI)   │
└──────────────────────┘      └──────────────────────────┘
            │                              │
            │  REST (/api/*)               │
            │  WebSocket (/collaboration/*)│
            │◄─────────────────────────────┤
            │                              │
            │  JSON responses using        │
            │  shared Document, Properties,
            │  User interfaces as the      │
            │  implicit contract           │
            │                              │
            ▼                              ▼
┌──────────────────────┐      ┌──────────────────────┐
│     PostgreSQL       │      │     Browser          │
│  ● documents table   │      │  ● IndexedDB (y-indexeddb)
│  ● document_associations     │  ● TanStack Query cache
│  ● users, workspaces │      │  ● TipTap + Yjs editor
└──────────────────────┘      └──────────────────────┘
```

Data Flow Through Shared Types

```
PostgreSQL row                  @ship/shared                    React component
──────────────                  ────────────                    ───────────────
documents.properties  ──────►  IssueProperties      ──────►  IssueSidebar.tsx
(JSONB blob)                   (TypeScript interface)         (typed rendering)

documents.document_type ────► DocumentType          ──────►  contextMenuActions.ts
(TEXT column)                  (union literal type)           (conditional menus)

session.created_at ──────────► SESSION_TIMEOUT_MS    ──────►  auth middleware (API)
(timestamp)                    (shared constant)              useSessionTimeout (Web)
```

The shared package serves as the type contract between API and web. The API writes data matching these interfaces into PostgreSQL; the web reads it back via REST and renders it using the same type definitions — ensuring both sides agree on shapes, allowed values, and business logic (like ICE score computation).

Phase 2

1. Tables & Relationships

17 tables total. Here's the map:

Core Content documents — The single unified table. All content types live here.

Columns: id, workspace_id, document_type (enum: wiki/issue/program/project/sprint/person/weekly_plan/weekly_retro/standup/weekly_review), title, content (JSONB TipTap), yjs_state (BYTEA CRDT), parent_id (self-FK for hierarchy), properties (JSONB for type-specific data), ticket_number, visibility, lifecycle timestamps (started_at, completed_at, cancelled_at, reopened_at), conversion tracking fields, soft-delete (deleted_at, archived_at) GIN index on properties for JSONB queries Partial index on properties->>'user_id' for person docs document_associations — Junction table for organizational relationships.

(document_id → documents, related_id → documents, relationship_type enum: parent/project/sprint/program) UNIQUE(document_id, related_id, relationship_type), CHECK no self-reference document_history — Field-level change audit trail.

(document_id → documents, field, old_value, new_value, changed_by → users, automated_by) document_links — Backlinks between documents.

(source_id → documents, target_id → documents), UNIQUE pair document_snapshots — Pre-conversion state for undo.

(document_id → documents, original document_type, title, properties, ticket_number, snapshot_reason) comments — Inline threaded comments on documents.

(document_id → documents, comment_id for thread grouping, parent_id → comments for replies, author_id → users, content, resolved_at)

Auth & Identity users — id, email (case-insensitive unique), password_hash (nullable for PIV-only), name, is_super_admin, last_workspace_id → workspaces, x509_subject_dn, last_auth_provider

sessions — id (hex string, not UUID), user_id → users, workspace_id → workspaces, expires_at, last_activity, user_agent, ip_address

oauth_state — Ephemeral OAuth flow tracking (state_id, nonce, code_verifier, expires_at)

api_tokens — CLI/programmatic access. token_hash (SHA-256), token_prefix (first 8 chars), user_id → users, workspace_id → workspaces, name, expires_at, revoked_at

Workspace & Access workspaces — id, name, sprint_start_date (drives all week calculations), archived_at

workspace_memberships — user_id → users, workspace_id → workspaces, role (admin/member). UNIQUE(workspace_id, user_id)

workspace_invites — email-based or PIV-based invites with token, expiration, x509_subject_dn

audit_logs — Compliance audit trail. actor_user_id, impersonating_user_id (for super-admin), action, resource_type, resource_id, details JSONB, ip_address, user_agent

Progress Tracking sprint_iterations — Claude Code /work progress. sprint_id → documents (week), story_id, status (pass/fail/in_progress), what_attempted, blockers_encountered, author_id → users

issue_iterations — Same structure but keyed to issue_id → documents instead of sprint

Files files — workspace_id → workspaces, uploaded_by → users, filename, mime_type, size_bytes, s3_key, cdn_url, status (pending/uploaded/failed)

Relationship Diagram

```
workspaces ──────────────────────────────────────────────────────────┐
  │ 1:N                                                              │
  ├──→ workspace_memberships ←── users (M:N through junction)        │
  ├──→ workspace_invites                                             │
  ├──→ audit_logs                                                    │
  ├──→ api_tokens ←── users                                          │
  ├──→ files ←── users                                               │
  ├──→ sessions ←── users                                            │
  ├──→ sprint_iterations ←── users                                   │
  ├──→ issue_iterations ←── users                                    │
  │                                                                  │
  └──→ documents ◄──────────────────────────────────────────────────┐│
        │                                                            ||
        ├──→ parent_id (self-FK, hierarchy)                          ||
        ├──→ converted_to_id / converted_from_id (self-FK)           ||
        │                                                            ||
        ├──→ document_associations (self-M:N junction)               ||
        │      document_id → documents                               ||
        │      related_id  → documents                               ||
        │      relationship_type: program|project|sprint|parent      ||
        │                                                            ||
        ├──→ document_history (1:N, field-level changes)             ||
        ├──→ document_links (self-M:N, backlinks)                    ||
        ├──→ document_snapshots (1:N, conversion undo)               ||
        ├──→ comments (1:N, threaded)                                ||
        ├──→ sprint_iterations (1:N, week progress)                  ||
        └──→ issue_iterations (1:N, issue progress)                  ||
                                                                     |
users ──────────────────────────────────────────────────────────────┘
  │
  ├──→ documents.created_by
  ├──→ document_history.changed_by
  ├──→ comments.author_id
  └──→ oauth_state (no FK, ephemeral)
```

The key takeaway: documents is the gravitational center. Almost every other table either references it directly or exists to support it. The document_associations junction table is the only way organizational relationships (program/project/week membership) are expressed — all legacy FK columns were dropped.

2. How one table serves all content types

The documents table has a fixed set of columns shared by everything (id, workspace_id, title, content, yjs_state, parent_id, position, ticket_number, timestamps, lifecycle dates) plus a properties JSONB column that holds type-specific data. An issue's state, priority, and assignee_id live in properties. A program's color and prefix live in properties. A week's sprint_number and owner_id live in properties.

The database doesn't enforce property structure — TypeScript does. Each document type has a corresponding interface (IssueProperties, WeekProperties, etc.) defined in shared/src/types/document.ts. The database just stores raw JSONB; the application layer casts and validates.

Every document type also gets the same TipTap rich-text editor, the same Yjs collaboration, the same 4-panel layout. The difference between an "issue" and a "wiki page" is which properties the sidebar shows and which workflows apply — not the underlying storage.

3. The document_type discriminator

document_type is a TEXT column with values: wiki, issue, program, project, sprint, person, weekly_plan, weekly_retro, standup, weekly_review.

In queries, it's a simple WHERE filter:

```
-- Get all issues in a workspace
SELECT * FROM documents WHERE workspace_id = $1 AND document_type = 'issue'

-- Get all weeks for a program (via association)
SELECT d.* FROM documents d
  JOIN document_associations da ON da.document_id = d.id
  WHERE da.related_id = $1 AND da.relationship_type = 'program'
    AND d.document_type = 'sprint'
```

In the frontend, it determines which sidebar component to render, which header badge to show, which placeholder text to use, and which room prefix (doc:, issue:, project:) the Yjs collaboration connects to. The DocumentType union type from @ship/shared gates all of this in TypeScript.

4. How document relationships work

Three mechanisms, each for a different purpose:

parent_id column — strict containment (1:1)

Used when a child document can't exist without its parent. Examples:

- Weekly plan → week document
- Weekly retro → week document
- Nested wiki page → parent wiki page

document_associations junction table — organizational (many-to-many)

Schema: (document_id, related_id, relationship_type) where relationship_type is 'program', 'project', or 'sprint'. Used when a document belongs to another for organizational grouping but could move. Examples:

- Issue belongs to a program
- Issue belongs to a project
- Issue assigned to a week

All three legacy FK columns (program_id, project_id, sprint_id) were dropped in migrations 027 and 029 after discovering write-read mismatch bugs. Now all reads and writes go through the junction table via utility functions in api/src/utils/document-crud.ts (addBelongsToAssociation, updateProgramAssociation, getBelongsToAssociationsBatch, etc.).

Implicit relationships via properties — for references that don't need a join table:

- WeekProperties.owner_id → points to a user
- PersonProperties.user_id → links person doc to auth user
- IssueProperties.assignee_id → points to a user

---

5. Request Flow: Creating an Issue

Full trace (click → database → UI update):

Step	Layer	File	What Happens
1	Click	React	IssuesList.tsx — handleCreateIssue() builds belongs_to[] from current context (program, project, sprint)
2	Mutation	React Query	useIssuesQuery.ts — createIssueMutation.mutateAsync() adds optimistic temp issue to cache immediately
3	HTTP	Fetch	api.ts — apiPost('/api/issues', {title: "Untitled", belongs_to}) fetches CSRF token first, sends with X-CSRF-Token header + session cookie
4	Validation	Express	routes/issues.ts — Zod schema validates body (title, state, priority, belongs_to, etc.)
5	Transaction	PostgreSQL	routes/issues.ts — BEGIN → advisory lock per workspace → MAX(ticket_number)+1 → INSERT INTO documents → INSERT INTO document_associations for each belongs_to → COMMIT
6	Response	Express	routes/issues.ts — 201 with full issue JSON including display_id: "#42" and enriched belongs_to (with titles/colors)
7	Cache update	React Query	useIssuesQuery.ts — onSuccess replaces optimistic issue with real data; onSettled invalidates all issue queries
8	Navigate	React Router	IssuesList.tsx — navigate(`/documents/${issue.id}`) opens the editor

Key detail — ticket number generation: Uses pg_advisory_xact_lock per workspace to serialize ticket number assignment. No sequence, no race condition.

6. Middleware Chain

Every request passes through these layers in app.ts, in order:

#	Middleware	Purpose
1	Trust Proxy	Recognizes CloudFront headers (prod only)
2	Helmet	Security headers: CSP, HSTS (1yr), X-Frame-Options, etc.
3	Rate Limiter	100 req/min prod, 1000 dev, 10000 test
4	CORS	Credentials enabled, origin: localhost:5173
5	JSON Parser	10MB max body
6	Cookie Parser	Signed cookies with session secret
7	Express Session	httpOnly, sameSite: strict, 15-min maxAge

Then per-route middleware:

#	Middleware	Applied To
8	CSRF (csrf-sync)	All state-changing routes — skipped for Bearer token auth
9	Auth	All protected routes — validates session OR API token
10	Authorization	Role-specific routes (superAdmin, workspaceAdmin, workspaceAccess)

Login-specific: /api/auth/login has its own rate limiter: 5 failed attempts per 15 minutes (skipSuccessfulRequests: true).

7. Authentication Flow

Two authentication paths:

Path A — Session Cookie (browser)

- Read session_id cookie → query sessions table (joined to users)
- Check absolute timeout (12 hours from creation) — delete + 401 if exceeded
- Check inactivity timeout (15 min since last_activity) — delete + 401 if exceeded
- Check workspace membership — delete + 403 if revoked
- Update last_activity; refresh cookie if >60s since last refresh (sliding window)
- Set req.userId, req.workspaceId, req.isSuperAdmin

Path B — Bearer Token (API/CLI)

- Read Authorization: Bearer <token> header
- SHA-256 hash the token → query api_tokens table by token_hash

- Check revoked_at is null, expires_at is in future
- Update last_used_at
- Set req.userId, req.workspaceId, req.isApiToken = true

Unauthenticated request outcomes

Condition	Status	Message
No cookie, no Bearer header	401	"No session found"
Invalid/unknown session ID	401	"Invalid session"
12-hour absolute timeout	401	"Session expired" (session deleted from DB)
15-min inactivity timeout	401	"Session expired due to inactivity" (session deleted)
Workspace membership revoked	403	"Access to this workspace has been revoked" (session deleted)
Invalid API token	401	"Unauthorized"

Session creation (on login): generates crypto.randomBytes(32).toString('hex'), inserts into sessions table with user_agent + ip_address for audit binding, sets httpOnly/secure/sameSite:strict cookie. Old session is deleted first (session fixation prevention).

Phase 2: Deep Dive

1. Real-time Collaboration

WebSocket establishment

- Client creates WebsocketProvider with connect: false, then calls connect() after setup
- URL pattern: ws(s)://{host}/collaboration/{roomPrefix}:{documentId} (e.g., doc:abc123, issue:xyz789)
- Server validates session cookie on upgrade; rejects with 401 if invalid
- Rate limited: 30 connections/min/IP, 50 messages/sec/connection

Yjs sync protocol

- Binary message types: messageSync=0, messageAwareness=1, messageClearCache=3
- On connect: server sends full Yjs state from PostgreSQL (yjs_state column)
- Incremental updates exchanged as Yjs update messages (CRDT diffs)

- Awareness protocol broadcasts cursor positions and user presence

Concurrent editing

- Yjs CRDTs handle conflict resolution automatically — no server-side merge logic needed
- Each client maintains a local Yjs doc; updates merge deterministically regardless of order
- Awareness state shows colored cursors per user (name + color stored in awareness)

Persistence

- Debounced 2-second write: after last update, waits 2s then persists merged Yjs state to documents.yjs_state (binary) and extracts TipTap JSON to documents.content
- On disconnect (last client leaves room): immediate final persist, then room cleanup
- Client-side: y-indexeddb caches Yjs state locally for instant load (300ms timeout before falling back to network)

2. TypeScript Patterns

Config: TypeScript with strict: true, noUncheckedIndexedAccess: true across all packages. Shared package compiles to both ESM and CJS via tsup.

Discriminated unions (primary pattern)

```
// DocumentType = 'wiki' | 'issue' | 'program' | 'project' | 'sprint' | 'person' | ...
// Each type has typed properties:
type WikiDocument = Document & { document_type: 'wiki'; properties: WikiProperties }
type IssueDocument = Document & { document_type: 'issue'; properties: IssueProperties }
```

Type guards

```
function isIssueDocument(doc: Document): doc is IssueDocument {
  return doc.document_type === 'issue';
}
```

Utility types: Partial<>, Pick<>, Omit<> used throughout for API request/response shaping. Record<string, unknown> for JSONB columns.

Generics: API route handlers use generic response types. TanStack Query hooks are typed with useQuery.

Shared types pattern: shared/src/types/document.ts exports ~48 types consumed by both api/ and web/ via workspace:* dependency. Ensures API responses and frontend

consumers agree on shape.

3. Testing Infrastructure

Structure

- Unit tests: Vitest in api/src/**/*.test.ts — run with pnpm test
- E2E tests: Playwright in e2e/tests/**/*.spec.ts — run via /e2e-test-runner skill (never directly)

Playwright fixtures (e2e/fixtures/isolated-env.ts)

Per-worker isolation: Each Playwright worker gets its own:

- PostgreSQL container (via testcontainers)
- API server (runs built dist/, not dev mode)
- Vite preview server (lightweight, ~40MB vs ~400MB for vite dev)
- Seed data: 1 workspace, 2 users, 5 programs, sprints, 24+ issues, projects, wikis

DB setup/teardown

- globalSetup: Builds API and web (pnpm build:api && pnpm build:web)
- Per-worker: Starts fresh PostgreSQL container → runs migrations → seeds data
- Teardown: Stops containers, kills server processes
- Dynamic worker count based on available memory (500MB per worker, 2GB reserved for OS)

Key config: 60s test timeout, 2 retries in CI / 1 local, line reporter for real-time progress.

4. Build and Deploy

Dockerfile

- Base: node:20-slim
- Production build: copies pre-built dist/, installs production deps only
- Entrypoint: runs migrations then starts Express on port 80
- No docker-compose for production. Local dev uses native PostgreSQL (not Docker). Testcontainers used only for E2E test isolation.

Terraform IaC (terraform/)

- VPC: Standard AWS networking
- Aurora PostgreSQL 16.8 Serverless v2: Auto-scaling database
- Elastic Beanstalk: Docker platform, t3.small, 1-4 instances, rolling deploys

- S3 + CloudFront: SPA hosting with WebSocket passthrough (explicit cache behaviors for /collaboration/* and /events)
- WAF: Optional

CI/CD (deploy scripts)

- ./scripts/deploy.sh prod — Backend → EB (zip + upload + deploy)
- ./scripts/deploy-frontend.sh prod — Frontend → S3 + CloudFront invalidation
- No GitHub Actions — deploy scripts run manually or via /workflows:deploy
- Shadow (UAT) environment deploys from feature branch before merge to master

---

Phase 3: Architecture Assessment

1. Strongest Decisions

- **Unified Document Model** — Single documents table with document_type discriminator and properties JSONB eliminates table sprawl. Adding a new content type requires zero schema migrations — just a new document_type value and TypeScript interface. This is the architectural backbone that makes the system extensible.

- **Yjs CRDTs for collaboration** — Choosing Yjs over operational transform (OT) means conflict resolution is mathematically guaranteed, works offline-tolerant, and requires no server-side merge logic. Combined with the shared Editor component pattern, every document type gets real-time collaboration for free.

- **E2E-first testing with per-worker isolation** — Each Playwright worker gets a fully isolated stack (own database, own API, own frontend). Tests are deterministic, parallelizable, and test real user flows. This catches integration bugs that unit tests miss.

2. Weakest Points

- **Raw SQL without an ORM or query builder** — Using pg directly means no type-safe queries, manual SQL string construction, and easy-to-miss migration issues. As the schema grows, this becomes increasingly error-prone. A lightweight query builder (Kysely, Drizzle) would add safety without ORM bloat.

- **Manual deployment scripts** — No CI/CD pipeline (GitHub Actions, etc.). Deploys are manual script runs, which risks human error, lacks audit trail, and doesn't enforce build/test gates before production. Fine for a small team, risky as the team scales.

- **Session-based auth with 15-minute timeout** — Aggressive for a productivity tool. Users editing long documents could lose their session mid-work. The session architecture also complicates horizontal scaling (sessions stored in PostgreSQL, every request hits DB for validation).

3. Onboarding Advice

Start here in this order:

- docs/unified-document-model.md — understand the single-table design
- docs/document-model-conventions.md — learn the 4-panel layout and terminology
- shared/src/types/document.ts — see how types flow between packages
- web/src/components/Editor.tsx — understand the shared editor pattern
- api/src/collaboration/index.ts — understand real-time sync
- Run pnpm dev and create a document — see all layers working together

4. Scaling Bottlenecks

- **WebSocket server is single-process** — All collaboration connections go to one Node.js process. Beyond ~1000 concurrent editors, you'd need Redis pub/sub or a dedicated Yjs server cluster to distribute rooms across instances.

- **PostgreSQL for everything** — Sessions, documents, Yjs state, file metadata, audit logs all in one Aurora instance. The yjs_state binary column grows with edit history. At scale, Yjs state should move to object storage (S3) with only metadata in PostgreSQL.

- **No background job system** — Document snapshots, audit log cleanup, and analytics are all synchronous. A job queue (BullMQ, SQS) would be needed for async processing at scale.

- **CloudFront WebSocket limitation** — WebSocket connections through CloudFront have a 24-hour idle timeout and add latency. At scale, direct ALB connections or a dedicated WebSocket endpoint would perform better.